BÀI TẬP TỔNG HỢP HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

Bài tập 1.

Cho lược đồ quan hệ sau:

XUONGPHIM(TenXuong,Diachi)

PHIM(MaPhim,TenPhim, NamSX, ThoiLuong, TenXuong)

DIENVIEN(MaDV,TenDV,Diachi, Gioitinh,NSinh)

DIENXUAT(MaPhim, MaDV)

Quan hệ	Thuộc tính	Kiểu dữ liệu	Diễn giải
XUONGPHIM	TenXuong	char(20)	Tên xưởng phim
	Diachi	varchar(50)	Địa chỉ của xưởng phim
PHIM	MaPhim	Char(10)	Mã phim
	TenPhim	char(20)	Tên phim
	NamSX	smalldatetime	Năm sản xuất phim
	ThoiLuong	int	Thời lượng phim (phút)
	TenXuong	char(20)	Tên xưởng phim sản xuất
DIENVIEN	TenDV	char(40)	Tên diễn viên
	Diachi	varchar(50)	Địa chỉ của diễn viên
	Giới tính	char(3)	Giới tính của diễn viên
	Nsinh	smalldatetime	Ngày sinh của diễn viên
DIENXUAT	MaPhim	char(10)	Mã phim
	TenDV	char(40)	Tên diễn viên

1. Lập trình cơ sở dữ liệu (T-SQL)

Câu 1. Viết câu lệnh liệt kê toàn bộ dữ liệu của bảng PHIM, sắp xếp tăng dần theo TenPhim.

Câu 2. Viết câu lệnh liệt kê các phim có thời lượng phim lớn hơn một giá trị do người dùng nhập vào (dùng biến).

Câu 3. Viết câu lệnh liệt kê phim cùng với thông tin xưởng phim tương ứng (kết hợp PHIM và XUONGPHIM bằng JOIN).

Câu 4. Viết câu lệnh thống kê số lượng phim theo từng XUONGPHIM, chỉ hiển thị những nhóm có số lượng lớn hơn 1 (dùng GROUP BY, HAVING).

Câu 5. Viết câu lệnh tìm phim có thời lượng phim cao nhất bằng subquery (không dùng TOP/MAX trực tiếp trong mệnh đề chính).

Câu 6. Viết câu lệnh sử dụng CTE (Common Table Expression) hoặc bảng tạm để tính thời lượng phim trung bình của từng XUONGPHIM rồi lọc ra những xưởng phim có giá trị trung bình cao hơn trung bình chung.

Câu 7. Viết đoạn T-SQL dùng con trỏ (CURSOR) hoặc vòng lặp WHILE để duyệt qua từng dòng của bảng PHIM và in ra thông tin TenPhim kèm thời lượng phim.

2. Ràng buộc dữ liệu (Constraints)

Câu 1. Thêm ràng buộc PRIMARY KEY cho bảng PHIM (nếu khóa chính chưa được khai báo khi tạo bảng).

Câu 2. Thêm ràng buộc FOREIGN KEY cho khóa ngoại của bảng XUONGPHIM tham chiếu đến bảng PHIM, quy định hành vi ON DELETE/ON UPDATE phù hợp.

Câu 3. Thêm ràng buộc CHECK cho cột ThoiLuong của bảng PHIM để đảm bảo giá trị luôn lớn hơn 0.

Câu 4. Thêm ràng buộc UNIQUE cho cột TenPhim của bảng PHIM để đảm bảo không trùng lặp.

Câu 5. Thêm ràng buộc DEFAULT cho một cột kiểu ngày (NamSX) của bảng PHIM, giá trị mặc định là ngày hiện tại (GETDATE()).

Câu 6. Viết câu lệnh ALTER TABLE để đổi tên ràng buộc CHECK vừa tạo và kiểm tra lại bằng cách chèn một dòng dữ liệu vi phạm ràng buộc để xác nhận lỗi được báo.

Câu 7. Thiết kế và cài đặt ràng buộc toàn vẹn phức tạp hơn bằng TRIGGER hoặc CHECK CONSTRAINT kết hợp hàm do người dùng định nghĩa, đảm bảo dữ liệu của bảng DIENXUAT luôn hợp lệ về mặt nghiệp vụ (ví dụ: một diễn viên không được diễn xuất trong hai phim có lịch chiếu trùng nhau).

3. View

Câu 1. Tạo view hiển thị danh sách phim gồm các cột TenPhim và ThoiLuong.

Câu 2. Tạo view kết hợp bảng PHIM và XUONGPHIM để hiển thị thông tin đầy đủ của từng phim kèm theo xưởng phim liên quan.

Câu 3. Tạo view thống kê tổng/đếm số lượng theo từng XUONGPHIM (sử dụng GROUP BY trong view).

Câu 4. Tạo view chỉ hiển thị những phim thỏa điều kiện nghiệp vụ cụ thể (ví dụ thời lượng phim lớn hơn một ngưỡng cho trước).

Câu 5. Viết câu lệnh cập nhật dữ liệu thông qua view vừa tạo ở câu 1 và giải thích điều kiện để một view có thể cập nhật được (updatable view).

Câu 6. Tạo view sử dụng WITH CHECK OPTION để ràng buộc dữ liệu cập nhật qua view phải thỏa điều kiện lọc của view.

Câu 7. Tạo view sử dụng hàm cửa sổ (window function) như ROW_NUMBER() hoặc RANK() để xếp hạng các phim theo thời lượng phim.

4. Stored Procedure

Câu 1. Viết stored procedure nhận vào mã của một phim và trả về đầy đủ thông tin của phim đó.

Câu 2. Viết stored procedure thêm mới một phim với các tham số đầu vào tương ứng các cột của bảng PHIM.

Câu 3. Viết stored procedure cập nhật ThoiLuong của một phim theo mã truyền vào, có kiểm tra dữ liệu tồn tại trước khi cập nhật.

Câu 4. Viết stored procedure xóa một phim theo mã, có xử lý ràng buộc khóa ngoại (kiểm tra tồn tại dữ liệu liên quan ở bảng DIENXUAT trước khi xóa).

Câu 5. Viết stored procedure có tham số OUTPUT trả về tổng số lượng hoặc tổng thời lượng phim của các phim thuộc một XUONGPHIM cụ thể.

Câu 6. Viết stored procedure thống kê danh sách phim theo điều kiện lọc động (tham số có thể NULL để bỏ qua điều kiện lọc đó).

Câu 7. Viết stored procedure sử dụng TRY...CATCH để xử lý lỗi khi thêm/xóa/cập nhật dữ liệu, kèm RAISERROR hoặc THROW để báo lỗi nghiệp vụ.

5. Function

Câu 1. Viết scalar function nhận vào mã phim và trả về TenPhim tương ứng.

Câu 2. Viết scalar function tính tuổi hoặc thời gian (theo năm sản xuất) tính đến thời điểm hiện tại của một phim.

Câu 3. Viết scalar function tính tổng thời lượng phim của các phim thuộc một XUONGPHIM được truyền vào làm tham số.

Câu 4. Viết table-valued function trả về danh sách phim thuộc một XUONGPHIM cụ thể (truyền mã XUONGPHIM làm tham số).

Câu 5. Viết table-valued function trả về danh sách phim có thời lượng phim nằm trong một khoảng giá trị (truyền 2 tham số min, max).

Câu 6. Sử dụng function vừa viết ở câu 4 kết hợp CROSS APPLY/OUTER APPLY để liệt kê thông tin của từng xưởng phim kèm danh sách phim tương ứng.

Câu 7. Viết function trả về bảng thống kê (nhiều dòng) gồm XUONGPHIM, số lượng phim và giá trị trung bình thời lượng phim theo từng XUONGPHIM.

6. Transaction

Câu 1. Viết đoạn lệnh sử dụng BEGIN TRANSACTION/COMMIT để thêm đồng thời một dòng vào bảng PHIM và một dòng liên quan vào bảng DIENXUAT, đảm bảo tính toàn vẹn (cả hai cùng thành công hoặc cùng thất bại).

Câu 2. Viết đoạn lệnh minh họa ROLLBACK TRANSACTION khi phát hiện lỗi trong quá trình cập nhật ThoiLuong của bảng PHIM.

Câu 3. Viết stored procedure thực hiện một giao dịch gồm nhiều bước (ví dụ: cập nhật bảng PHIM và bảng XUONGPHIM), sử dụng TRY...CATCH kết hợp TRANSACTION để đảm bảo an toàn dữ liệu.

Câu 4. Trình bày và minh họa bằng ví dụ SQL các mức cô lập giao dịch (isolation level) READ COMMITTED và SERIALIZABLE khi nhiều người dùng cùng truy cập bảng PHIM.

Câu 5. Viết đoạn lệnh sử dụng SAVE TRANSACTION (savepoint) để chỉ rollback một phần thao tác trong một giao dịch gồm nhiều lệnh INSERT/UPDATE trên bảng PHIM và XUONGPHIM.

Câu 6. Mô phỏng tình huống deadlock giữa hai giao dịch cùng thao tác trên bảng PHIM và XUONGPHIM theo thứ tự ngược nhau, giải thích cách SQL Server phát hiện và xử lý deadlock.

7. Trigger

Câu 1. Viết trigger AFTER INSERT trên bảng PHIM để tự động ghi log (thêm dòng vào bảng lịch sử) mỗi khi có phim mới được thêm.

Câu 2. Viết trigger INSTEAD OF DELETE trên bảng PHIM để ngăn không cho xóa phim khi vẫn còn dữ liệu liên quan ở bảng DIENXUAT.

Câu 3. Viết trigger AFTER UPDATE trên bảng PHIM để kiểm tra không cho phép cập nhật ThoiLuong thành giá trị âm, nếu vi phạm thì ROLLBACK.

Câu 4. Viết trigger trên bảng DIENXUAT để tự động cập nhật (tính toán lại) một giá trị tổng hợp ở bảng PHIM hoặc XUONGPHIM mỗi khi có thay đổi dữ liệu ở bảng DIENXUAT.

Câu 5. Viết trigger sử dụng bảng ảo inserted và deleted để so sánh giá trị cũ và giá trị mới của ThoiLuong khi cập nhật bảng PHIM, chỉ cho phép tăng chứ không cho phép giảm.

Câu 6. Viết trigger DDL (ví dụ: ON DATABASE FOR DROP_TABLE) để ngăn việc xóa nhầm các bảng quan trọng trong cơ sở dữ liệu QL_XuongPhim.

Câu 7. Viết trigger để đảm bảo ràng buộc nghiệp vụ liên quan đến một diễn viên không được diễn xuất trong hai phim có lịch chiếu trùng nhau, tự động chặn hoặc điều chỉnh dữ liệu khi vi phạm.

8. Index (Chỉ mục)

Câu 1. Tạo chỉ mục (index) cho cột khóa ngoại của bảng XUONGPHIM tham chiếu đến bảng PHIM nhằm tăng tốc độ JOIN.

Câu 2. Tạo UNIQUE INDEX cho cột TenPhim của bảng PHIM để vừa đảm bảo tính duy nhất vừa tăng tốc tìm kiếm.

Câu 3. Tạo chỉ mục kết hợp (composite index) trên hai cột thường được dùng chung trong điều kiện WHERE của bảng PHIM (ví dụ TenPhim và ThoiLuong).

Câu 4. Sử dụng câu lệnh EXPLAIN/hiển thị execution plan (SET STATISTICS IO ON, hoặc xem Actual Execution Plan) để so sánh hiệu năng truy vấn trên bảng PHIM trước và sau khi tạo chỉ mục.

Câu 5. Tạo chỉ mục lọc (filtered index) trên bảng PHIM chỉ đánh chỉ mục cho những dòng thỏa một điều kiện cụ thể.

Câu 6. Trình bày sự khác nhau giữa clustered index và non-clustered index, minh họa bằng cách tạo cả hai loại trên bảng PHIM.

Câu 7. Viết câu lệnh xóa (DROP INDEX) và tạo lại (REBUILD/REORGANIZE) chỉ mục trên bảng PHIM, giải thích khi nào nên bảo trì lại chỉ mục.

9. Role (Vai trò)

Câu 1. Tạo một login mới ở mức SQL Server và một user tương ứng trong cơ sở dữ liệu QL_XuongPhim.

Câu 2. Tạo một database role (ví dụ role_XemPhim) chỉ có quyền đọc dữ liệu (SELECT) trên các bảng PHIM và XUONGPHIM, sau đó thêm user vừa tạo vào role này.

Câu 3. Tạo một database role (ví dụ role_QuanLyPhim) có quyền INSERT, UPDATE trên bảng PHIM nhưng không có quyền DELETE.

Câu 4. Sử dụng lệnh sp_addrolemember (hoặc ALTER ROLE ... ADD MEMBER) để gán user vào nhiều role khác nhau, sau đó kiểm tra quyền hạn bằng lệnh EXECUTE AS USER.

Câu 5. Tạo một application role cho ứng dụng quản lý QL_XuongPhim, minh họa cách kích hoạt application role bằng sp_setapprole.

Câu 6. Viết câu lệnh loại một user ra khỏi role và xóa role khi không còn sử dụng, kiểm tra quyền của user sau khi bị loại khỏi role.

10. Bảo mật (Security)

Câu 1. Viết câu lệnh GRANT cấp quyền SELECT trên bảng PHIM cho một user cụ thể.

Câu 2. Viết câu lệnh DENY quyền DELETE trên bảng PHIM cho một user hoặc role cụ thể, giải thích thứ tự ưu tiên giữa GRANT, DENY và REVOKE.

Câu 3. Viết câu lệnh REVOKE để thu hồi lại quyền đã cấp ở câu 1, kiểm tra lại quyền của user bằng cách đăng nhập thử (hoặc EXECUTE AS).

Câu 4. Cấp quyền EXECUTE cho một stored procedure đã tạo ở phần trước cho một role cụ thể, thay vì cấp quyền trực tiếp trên bảng.

Câu 5. Trình bày và minh họa cách mã hóa dữ liệu nhạy cảm trong bảng PHIM (ví dụ dùng ENCRYPTBYPASSPHRASE/DECRYPTBYPASSPHRASE) đối với cột lưu thông tin cần bảo mật.

Câu 6. Thiết lập column-level security: chỉ cấp quyền SELECT trên một số cột cụ thể của bảng PHIM (không cấp toàn bộ cột) cho một user.

Câu 7. Tạo row-level security (Security Policy với Security Predicate) để mỗi user chỉ xem được các dòng dữ liệu của bảng PHIM thuộc phạm vi quản lý của mình (ví dụ theo XUONGPHIM).

11. BackUp & Restore

Câu 1. Chuẩn bị và sao lưu Full

1. Xác nhận recovery model của <CSDL> là FULL (truy vấn sys.databases), đổi nếu cần.
2. Ghi lại số liệu mốc của các <bảng chính> (số dòng, tổng một cột số nếu có) để đối chiếu về sau.
3. Thực hiện full backup có CHECKSUM ra file theo quy ước đặt tên.
4. Kiểm tra file bằng RESTORE HEADERONLY, RESTORE FILELISTONLY, RESTORE VERIFYONLY; ghi lại tên logic và đường dẫn các file dữ liệu/log.

Kết quả: script, bảng số liệu mốc, ảnh kết quả kiểm tra file backup.

Câu 2. Sao lưu Differential và Log

1. Thực hiện lần lượt: thay đổi dữ liệu #1 → differential backup → thay đổi #2 → log backup 1 → thay đổi #3 → log backup 2 (nội dung các thay đổi nêu trong tình huống).
2. Truy vấn msdb.dbo.backupset và msdb.dbo.backupmediafamily liệt kê loại backup, thời điểm, first_lsn, last_lsn, đường dẫn file.
3. Chứng minh chuỗi log liên tục (first_lsn của log sau khớp last_lsn của log trước).

Kết quả: script, kết quả truy vấn msdb kèm nhận xét về chuỗi LSN.

Câu 3. Restore chuỗi backup sang CSDL mới

1. Restore full → differential → log1 → log2 sang CSDL mới <CSDL>_Test bằng MOVE, không ghi đè CSDL gốc.
2. Đối chiếu số liệu của các <bảng chính> giữa hai CSDL theo kỳ vọng trong tình huống.
3. Giải thích vì sao dùng NORECOVERY ở các bước trung gian và RECOVERY ở bước cuối.

Kết quả: script, bảng đối chiếu, phần giải thích.

Câu 4. Khôi phục sau sự cố logic (xóa hoặc sửa nhầm)

1. Mô phỏng đúng sự cố nêu trong tình huống; ghi lại thời điểm ngay trước sự cố.
2. Thực hiện phương pháp khôi phục chỉ định trong tình huống: **(a)** tail-log backup rồi restore đè với STOPAT; **(b)** restore sang CSDL tạm rồi sửa lại dữ liệu cho CSDL gốc; hoặc **(c)** dùng STOPBEFOREMARK.
3. Đối chiếu với số liệu mốc để xác nhận dữ liệu đã trở về đúng trạng thái trước sự cố.
4. Nêu các giao dịch hợp lệ (nếu có) phát sinh sau thời điểm sự cố và cách xử lý chúng.

Kết quả: script, ảnh kết quả trước/sau, phần nhận xét.

Câu 5. Khôi phục sau mất file dữ liệu

1. Thêm dữ liệu mới nêu trong tình huống và COMMIT **sau log backup cuối** (chưa có bản backup nào chứa dữ liệu này).
2. Giả lập mất file dữ liệu: đưa CSDL OFFLINE, đổi tên file .mdf, cho ONLINE lại và quan sát trạng thái trong sys.databases.
3. Thực hiện tail-log backup bằng WITH NO_TRUNCATE, rồi khôi phục toàn bộ chuỗi backup.
4. Kiểm tra dữ liệu mới vẫn còn; giải thích nguyên nhân (liên hệ Write-Ahead Logging).

Kết quả: script, ảnh trạng thái CSDL trước/sau, phần giải thích.

Bài tập 2.

Cho lược đồ quan hệ sau:

NHANVIEN(MaNV, TenNV, MaVT, MaPB, Luong, NgayVL)

PHONGBAN(MaPB, TenPB, Diachi,NgayTL, MaNV)

VITRI(MaVT, Mota)

MUCLUONG(MaML, Mucthap, Muccao)

Quan hệ	Thuộc tính	Kiểu dữ liệu	Diễn giải
NHANVIEN	MaNV	int	Mã nhân viên
	TenNV	varchar(30)	Tên nhân viên
	MaVT	int	Mã vị trí làm việc của nhân viên
	MaPB	int	Mã phòng ban của nhân viên
	Luong	int	Lương của nhân viên
	NgayVL	smalldatetime	Ngày vào làm của nhân viên
PHONGBAN	MaPB	int	Mã phòng ban
	TenPB	char(10)	Tên phòng ban
	Diachi	varchar(40)	Địa chỉ phòng ban
	NgayTL	smalldatetime	Ngày thành lập của phòng ban.
	MaNV	int	Mã nhân viên làm trưởng phòng
VITRI	MaVT	int	Mã vị trí của nhân viên trong công ty
	Mota	varchar(30)	Tên vị trí
MUCLUONG	MaML	int	Mã mức lương
	Mucthap	int	Giá trị mức lương thấp nhất
	Muccao	int	Giá trị mức lương cao nhất

1. Lập trình cơ sở dữ liệu (T-SQL)

Câu 1. Viết câu lệnh liệt kê toàn bộ dữ liệu của bảng NHANVIEN, sắp xếp tăng dần theo TenNV.

Câu 2. Viết câu lệnh liệt kê các nhân viên có lương lớn hơn một giá trị do người dùng nhập vào (dùng biến).

Câu 3. Viết câu lệnh liệt kê nhân viên cùng với thông tin phòng ban tương ứng (kết hợp NHANVIEN và PHONGBAN bằng JOIN).

Câu 4. Viết câu lệnh thống kê số lượng nhân viên theo từng PHONGBAN, chỉ hiển thị những nhóm có số lượng lớn hơn 1 (dùng GROUP BY, HAVING).

Câu 5. Viết câu lệnh tìm nhân viên có lương cao nhất bằng subquery (không dùng TOP/MAX trực tiếp trong mệnh đề chính).

Câu 6. Viết câu lệnh sử dụng CTE (Common Table Expression) hoặc bảng tạm để tính lương trung bình của từng PHONGBAN rồi lọc ra những phòng ban có giá trị trung bình cao hơn trung bình chung.

Câu 7. Viết đoạn T-SQL dùng con trỏ (CURSOR) hoặc vòng lặp WHILE để duyệt qua từng dòng của bảng NHANVIEN và in ra thông tin TenNV kèm lương.

2. Ràng buộc dữ liệu (Constraints)

Câu 1. Thêm ràng buộc PRIMARY KEY cho bảng NHANVIEN (nếu khóa chính chưa được khai báo khi tạo bảng).

Câu 2. Thêm ràng buộc FOREIGN KEY cho khóa ngoại của bảng PHONGBAN tham chiếu đến bảng NHANVIEN, quy định hành vi ON DELETE/ON UPDATE phù hợp.

Câu 3. Thêm ràng buộc CHECK cho cột Luong của bảng NHANVIEN để đảm bảo giá trị luôn lớn hơn 0.

Câu 4. Thêm ràng buộc UNIQUE cho cột TenNV của bảng NHANVIEN để đảm bảo không trùng lặp.

Câu 5. Thêm ràng buộc DEFAULT cho một cột kiểu ngày (NgayVL) của bảng NHANVIEN, giá trị mặc định là ngày hiện tại (GETDATE()).

Câu 6. Viết câu lệnh ALTER TABLE để đổi tên ràng buộc CHECK vừa tạo và kiểm tra lại bằng cách chèn một dòng dữ liệu vi phạm ràng buộc để xác nhận lỗi được báo.

Câu 7. Thiết kế và cài đặt ràng buộc toàn vẹn phức tạp hơn bằng TRIGGER hoặc CHECK CONSTRAINT kết hợp hàm do người dùng định nghĩa, đảm bảo dữ liệu của bảng VITRI luôn hợp lệ về mặt nghiệp vụ (ví dụ: lương của nhân viên phải nằm trong khoảng mức lương (MUCLUONG) quy định cho vị trí của họ).

3. View

Câu 1. Tạo view hiển thị danh sách nhân viên gồm các cột TenNV và Luong.

Câu 2. Tạo view kết hợp bảng NHANVIEN và PHONGBAN để hiển thị thông tin đầy đủ của từng nhân viên kèm theo phòng ban liên quan.

Câu 3. Tạo view thống kê tổng/đếm số lượng theo từng PHONGBAN (sử dụng GROUP BY trong view).

Câu 4. Tạo view chỉ hiển thị những nhân viên thỏa điều kiện nghiệp vụ cụ thể (ví dụ lương lớn hơn một ngưỡng cho trước).

Câu 5. Viết câu lệnh cập nhật dữ liệu thông qua view vừa tạo ở câu 1 và giải thích điều kiện để một view có thể cập nhật được (updatable view).

Câu 6. Tạo view sử dụng WITH CHECK OPTION để ràng buộc dữ liệu cập nhật qua view phải thỏa điều kiện lọc của view.

Câu 7. Tạo view sử dụng hàm cửa sổ (window function) như ROW_NUMBER() hoặc RANK() để xếp hạng các nhân viên theo lương.

4. Stored Procedure

Câu 1. Viết stored procedure nhận vào mã của một nhân viên và trả về đầy đủ thông tin của nhân viên đó.

Câu 2. Viết stored procedure thêm mới một nhân viên với các tham số đầu vào tương ứng các cột của bảng NHANVIEN.

Câu 3. Viết stored procedure cập nhật Luong của một nhân viên theo mã truyền vào, có kiểm tra dữ liệu tồn tại trước khi cập nhật.

Câu 4. Viết stored procedure xóa một nhân viên theo mã, có xử lý ràng buộc khóa ngoại (kiểm tra tồn tại dữ liệu liên quan ở bảng VITRI trước khi xóa).

Câu 5. Viết stored procedure có tham số OUTPUT trả về tổng số lượng hoặc tổng lương của các nhân viên thuộc một PHONGBAN cụ thể.

Câu 6. Viết stored procedure thống kê danh sách nhân viên theo điều kiện lọc động (tham số có thể NULL để bỏ qua điều kiện lọc đó).

Câu 7. Viết stored procedure sử dụng TRY...CATCH để xử lý lỗi khi thêm/xóa/cập nhật dữ liệu, kèm RAISERROR hoặc THROW để báo lỗi nghiệp vụ.

5. Function

Câu 1. Viết scalar function nhận vào mã nhân viên và trả về TenNV tương ứng.

Câu 2. Viết scalar function tính tuổi hoặc thời gian (theo ngày vào làm) tính đến thời điểm hiện tại của một nhân viên.

Câu 3. Viết scalar function tính tổng lương của các nhân viên thuộc một PHONGBAN được truyền vào làm tham số.

Câu 4. Viết table-valued function trả về danh sách nhân viên thuộc một PHONGBAN cụ thể (truyền mã PHONGBAN làm tham số).

Câu 5. Viết table-valued function trả về danh sách nhân viên có lương nằm trong một khoảng giá trị (truyền 2 tham số min, max).

Câu 6. Sử dụng function vừa viết ở câu 4 kết hợp CROSS APPLY/OUTER APPLY để liệt kê thông tin của từng phòng ban kèm danh sách nhân viên tương ứng.

Câu 7. Viết function trả về bảng thống kê (nhiều dòng) gồm PHONGBAN, số lượng nhân viên và giá trị trung bình lương theo từng PHONGBAN.

6. Transaction

Câu 1. Viết đoạn lệnh sử dụng BEGIN TRANSACTION/COMMIT để thêm đồng thời một dòng vào bảng NHANVIEN và một dòng liên quan vào bảng VITRI, đảm bảo tính toàn vẹn (cả hai cùng thành công hoặc cùng thất bại).

Câu 2. Viết đoạn lệnh minh họa ROLLBACK TRANSACTION khi phát hiện lỗi trong quá trình cập nhật Luong của bảng NHANVIEN.

Câu 3. Viết stored procedure thực hiện một giao dịch gồm nhiều bước (ví dụ: cập nhật bảng NHANVIEN và bảng PHONGBAN), sử dụng TRY...CATCH kết hợp TRANSACTION để đảm bảo an toàn dữ liệu.

Câu 4. Trình bày và minh họa bằng ví dụ SQL các mức cô lập giao dịch (isolation level) READ COMMITTED và SERIALIZABLE khi nhiều người dùng cùng truy cập bảng NHANVIEN.

Câu 5. Viết đoạn lệnh sử dụng SAVE TRANSACTION (savepoint) để chỉ rollback một phần thao tác trong một giao dịch gồm nhiều lệnh INSERT/UPDATE trên bảng NHANVIEN và PHONGBAN.

Câu 6. Mô phỏng tình huống deadlock giữa hai giao dịch cùng thao tác trên bảng NHANVIEN và PHONGBAN theo thứ tự ngược nhau, giải thích cách SQL Server phát hiện và xử lý deadlock.

7. Trigger

Câu 1. Viết trigger AFTER INSERT trên bảng NHANVIEN để tự động ghi log (thêm dòng vào bảng lịch sử) mỗi khi có nhân viên mới được thêm.

Câu 2. Viết trigger INSTEAD OF DELETE trên bảng NHANVIEN để ngăn không cho xóa nhân viên khi vẫn còn dữ liệu liên quan ở bảng VITRI.

Câu 3. Viết trigger AFTER UPDATE trên bảng NHANVIEN để kiểm tra không cho phép cập nhật Luong thành giá trị âm, nếu vi phạm thì ROLLBACK.

Câu 4. Viết trigger trên bảng VITRI để tự động cập nhật (tính toán lại) một giá trị tổng hợp ở bảng NHANVIEN hoặc PHONGBAN mỗi khi có thay đổi dữ liệu ở bảng VITRI.

Câu 5. Viết trigger sử dụng bảng ảo inserted và deleted để so sánh giá trị cũ và giá trị mới của Luong khi cập nhật bảng NHANVIEN, chỉ cho phép tăng chứ không cho phép giảm.

Câu 6. Viết trigger DDL (ví dụ: ON DATABASE FOR DROP_TABLE) để ngăn việc xóa nhầm các bảng quan trọng trong cơ sở dữ liệu QL_NhanSu.

Câu 7. Viết trigger để đảm bảo ràng buộc nghiệp vụ liên quan đến lương của nhân viên phải nằm trong khoảng mức lương (MUCLUONG) quy định cho vị trí của họ, tự động chặn hoặc điều chỉnh dữ liệu khi vi phạm.

8. Index (Chỉ mục)

Câu 1. Tạo chỉ mục (index) cho cột khóa ngoại của bảng PHONGBAN tham chiếu đến bảng NHANVIEN nhằm tăng tốc độ JOIN.

Câu 2. Tạo UNIQUE INDEX cho cột TenNV của bảng NHANVIEN để vừa đảm bảo tính duy nhất vừa tăng tốc tìm kiếm.

Câu 3. Tạo chỉ mục kết hợp (composite index) trên hai cột thường được dùng chung trong điều kiện WHERE của bảng NHANVIEN (ví dụ TenNV và Luong).

Câu 4. Sử dụng câu lệnh EXPLAIN/hiển thị execution plan (SET STATISTICS IO ON, hoặc xem Actual Execution Plan) để so sánh hiệu năng truy vấn trên bảng NHANVIEN trước và sau khi tạo chỉ mục.

Câu 5. Tạo chỉ mục lọc (filtered index) trên bảng NHANVIEN chỉ đánh chỉ mục cho những dòng thỏa một điều kiện cụ thể.

Câu 6. Trình bày sự khác nhau giữa clustered index và non-clustered index, minh họa bằng cách tạo cả hai loại trên bảng NHANVIEN.

Câu 7. Viết câu lệnh xóa (DROP INDEX) và tạo lại (REBUILD/REORGANIZE) chỉ mục trên bảng NHANVIEN, giải thích khi nào nên bảo trì lại chỉ mục.

9. Role (Vai trò)

Câu 1. Tạo một login mới ở mức SQL Server và một user tương ứng trong cơ sở dữ liệu QL_NhanSu.

Câu 2. Tạo một database role (ví dụ role_XemNhanSu) chỉ có quyền đọc dữ liệu (SELECT) trên các bảng NHANVIEN và PHONGBAN, sau đó thêm user vừa tạo vào role này.

Câu 3. Tạo một database role (ví dụ role_QuanLyNhanSu) có quyền INSERT, UPDATE trên bảng NHANVIEN nhưng không có quyền DELETE.

Câu 4. Sử dụng lệnh sp_addrolemember (hoặc ALTER ROLE ... ADD MEMBER) để gán user vào nhiều role khác nhau, sau đó kiểm tra quyền hạn bằng lệnh EXECUTE AS USER.

Câu 5. Tạo một application role cho ứng dụng quản lý QL_NhanSu, minh họa cách kích hoạt application role bằng sp_setapprole.

Câu 6. Viết câu lệnh loại một user ra khỏi role và xóa role khi không còn sử dụng, kiểm tra quyền của user sau khi bị loại khỏi role.

10. Bảo mật (Security)

Câu 1. Viết câu lệnh GRANT cấp quyền SELECT trên bảng NHANVIEN cho một user cụ thể.

Câu 2. Viết câu lệnh DENY quyền DELETE trên bảng NHANVIEN cho một user hoặc role cụ thể, giải thích thứ tự ưu tiên giữa GRANT, DENY và REVOKE.

Câu 3. Viết câu lệnh REVOKE để thu hồi lại quyền đã cấp ở câu 1, kiểm tra lại quyền của user bằng cách đăng nhập thử (hoặc EXECUTE AS).

Câu 4. Cấp quyền EXECUTE cho một stored procedure đã tạo ở phần trước cho một role cụ thể, thay vì cấp quyền trực tiếp trên bảng.

Câu 5. Trình bày và minh họa cách mã hóa dữ liệu nhạy cảm trong bảng NHANVIEN (ví dụ dùng ENCRYPTBYPASSPHRASE/DECRYPTBYPASSPHRASE) đối với cột lưu thông tin cần bảo mật.

Câu 6. Thiết lập column-level security: chỉ cấp quyền SELECT trên một số cột cụ thể của bảng NHANVIEN (không cấp toàn bộ cột) cho một user.

Câu 7. Tạo row-level security (Security Policy với Security Predicate) để mỗi user chỉ xem được các dòng dữ liệu của bảng NHANVIEN thuộc phạm vi quản lý của mình (ví dụ theo PHONGBAN).

Bài tập 3.

Cho lược đồ Cơ sở dữ liệu quản lý đề án công ty như sau:

PHONGBAN (MAPHG, TENPHG, DIADIEM)

Tân từ: Công ty có nhiều phòng ban, mỗi phòng sẽ lưu trữ thông tin mã phòng, tên phòng, địa điểm của phòng.

NHANVIEN(MANV,HOTEN,NGSINH,GIOITINH,CHUCVU,MAPHG)

Tân từ: nhân viên có một mã số để phân biệt với các nhân viên khác, cần lưu trữ họ tên, ngày sinh, địa chỉ, ngày vào làm, hệ số lương, mức lương, chức vụ hiện tại và thuộc phòng ban nào.

DEAN (MADA, TENDA, NGBD, NGKT)

Tân từ: đề án có một mã số để phân biệt với các đề án khác, tên đề án, ngày bắt đầu thực hiện và ngày kết thúc.

PHANCONG (MANV,MADA,TUNGAY,DENNGAY)

Tân từ: mỗi đề án được phân thành nhiều giai đoạn (từ ngày đến ngày), mỗi nhân viên sẽ được phân công thực hiện các đề án ở theo từng giai đoạn.

1. Lập trình cơ sở dữ liệu (T-SQL)

Câu 1. Viết câu lệnh liệt kê toàn bộ dữ liệu của bảng NHANVIEN, sắp xếp tăng dần theo HOTEN.

Câu 2. Viết câu lệnh liệt kê các nhân viên có số đề án tham gia lớn hơn một giá trị do người dùng nhập vào (dùng biến).

Câu 3. Viết câu lệnh liệt kê nhân viên cùng với thông tin phòng ban tương ứng (kết hợp NHANVIEN và PHONGBAN bằng JOIN).

Câu 4. Viết câu lệnh thống kê số lượng nhân viên theo từng PHONGBAN, chỉ hiển thị những nhóm có số lượng lớn hơn 1 (dùng GROUP BY, HAVING).

Câu 5. Viết câu lệnh tìm nhân viên có số đề án tham gia cao nhất bằng subquery (không dùng TOP/MAX trực tiếp trong mệnh đề chính).

Câu 6. Viết câu lệnh sử dụng CTE (Common Table Expression) hoặc bảng tạm để tính số đề án tham gia trung bình của từng PHONGBAN rồi lọc ra những phòng ban có giá trị trung bình cao hơn trung bình chung.

Câu 7. Viết đoạn T-SQL dùng con trỏ (CURSOR) hoặc vòng lặp WHILE để duyệt qua từng dòng của bảng NHANVIEN và in ra thông tin HOTEN kèm số đề án tham gia.

2. Ràng buộc dữ liệu (Constraints)

Câu 1. Thêm ràng buộc PRIMARY KEY cho bảng NHANVIEN (nếu khóa chính chưa được khai báo khi tạo bảng).

Câu 2. Thêm ràng buộc FOREIGN KEY cho khóa ngoại của bảng PHONGBAN tham chiếu đến bảng NHANVIEN, quy định hành vi ON DELETE/ON UPDATE phù hợp.

Câu 3. Thêm ràng buộc CHECK cho cột MADA của bảng NHANVIEN để đảm bảo giá trị luôn lớn hơn 0.

Câu 4. Thêm ràng buộc UNIQUE cho cột HOTEN của bảng NHANVIEN để đảm bảo không trùng lặp.

Câu 5. Thêm ràng buộc DEFAULT cho một cột kiểu ngày (NGBD) của bảng NHANVIEN, giá trị mặc định là ngày hiện tại (GETDATE()).

Câu 6. Viết câu lệnh ALTER TABLE để đổi tên ràng buộc CHECK vừa tạo và kiểm tra lại bằng cách chèn một dòng dữ liệu vi phạm ràng buộc để xác nhận lỗi được báo.

Câu 7. Thiết kế và cài đặt ràng buộc toàn vẹn phức tạp hơn bằng TRIGGER hoặc CHECK CONSTRAINT kết hợp hàm do người dùng định nghĩa, đảm bảo dữ liệu của bảng PHANCONG luôn hợp lệ về mặt nghiệp vụ (ví dụ: một nhân viên không được phân công vào hai đề án có khoảng thời gian (TUNGAY–DENNGAY) chồng lấp nhau).

3. View

Câu 1. Tạo view hiển thị danh sách nhân viên gồm các cột HOTEN và MADA.

Câu 2. Tạo view kết hợp bảng NHANVIEN và PHONGBAN để hiển thị thông tin đầy đủ của từng nhân viên kèm theo phòng ban liên quan.

Câu 3. Tạo view thống kê tổng/đếm số lượng theo từng PHONGBAN (sử dụng GROUP BY trong view).

Câu 4. Tạo view chỉ hiển thị những nhân viên thỏa điều kiện nghiệp vụ cụ thể (ví dụ số đề án tham gia lớn hơn một ngưỡng cho trước).

Câu 5. Viết câu lệnh cập nhật dữ liệu thông qua view vừa tạo ở câu 1 và giải thích điều kiện để một view có thể cập nhật được (updatable view).

Câu 6. Tạo view sử dụng WITH CHECK OPTION để ràng buộc dữ liệu cập nhật qua view phải thỏa điều kiện lọc của view.

Câu 7. Tạo view sử dụng hàm cửa sổ (window function) như ROW_NUMBER() hoặc RANK() để xếp hạng các nhân viên theo số đề án tham gia.

4. Stored Procedure

Câu 1. Viết stored procedure nhận vào mã của một nhân viên và trả về đầy đủ thông tin của nhân viên đó.

Câu 2. Viết stored procedure thêm mới một nhân viên với các tham số đầu vào tương ứng các cột của bảng NHANVIEN.

Câu 3. Viết stored procedure cập nhật MADA của một nhân viên theo mã truyền vào, có kiểm tra dữ liệu tồn tại trước khi cập nhật.

Câu 4. Viết stored procedure xóa một nhân viên theo mã, có xử lý ràng buộc khóa ngoại (kiểm tra tồn tại dữ liệu liên quan ở bảng PHANCONG trước khi xóa).

Câu 5. Viết stored procedure có tham số OUTPUT trả về tổng số lượng hoặc tổng số đề án tham gia của các nhân viên thuộc một PHONGBAN cụ thể.

Câu 6. Viết stored procedure thống kê danh sách nhân viên theo điều kiện lọc động (tham số có thể NULL để bỏ qua điều kiện lọc đó).

Câu 7. Viết stored procedure sử dụng TRY...CATCH để xử lý lỗi khi thêm/xóa/cập nhật dữ liệu, kèm RAISERROR hoặc THROW để báo lỗi nghiệp vụ.

5. Function

Câu 1. Viết scalar function nhận vào mã nhân viên và trả về HOTEN tương ứng.

Câu 2. Viết scalar function tính tuổi hoặc thời gian (theo ngày bắt đầu đề án) tính đến thời điểm hiện tại của một nhân viên.

Câu 3. Viết scalar function tính tổng số đề án tham gia của các nhân viên thuộc một PHONGBAN được truyền vào làm tham số.

Câu 4. Viết table-valued function trả về danh sách nhân viên thuộc một PHONGBAN cụ thể (truyền mã PHONGBAN làm tham số).

Câu 5. Viết table-valued function trả về danh sách nhân viên có số đề án tham gia nằm trong một khoảng giá trị (truyền 2 tham số min, max).

Câu 6. Sử dụng function vừa viết ở câu 4 kết hợp CROSS APPLY/OUTER APPLY để liệt kê thông tin của từng phòng ban kèm danh sách nhân viên tương ứng.

Câu 7. Viết function trả về bảng thống kê (nhiều dòng) gồm PHONGBAN, số lượng nhân viên và giá trị trung bình số đề án tham gia theo từng PHONGBAN.

6. Transaction

Câu 1. Viết đoạn lệnh sử dụng BEGIN TRANSACTION/COMMIT để thêm đồng thời một dòng vào bảng NHANVIEN và một dòng liên quan vào bảng PHANCONG, đảm bảo tính toàn vẹn (cả hai cùng thành công hoặc cùng thất bại).

Câu 2. Viết đoạn lệnh minh họa ROLLBACK TRANSACTION khi phát hiện lỗi trong quá trình cập nhật MADA của bảng NHANVIEN.

Câu 3. Viết stored procedure thực hiện một giao dịch gồm nhiều bước (ví dụ: cập nhật bảng NHANVIEN và bảng PHONGBAN), sử dụng TRY...CATCH kết hợp TRANSACTION để đảm bảo an toàn dữ liệu.

Câu 4. Trình bày và minh họa bằng ví dụ SQL các mức cô lập giao dịch (isolation level) READ COMMITTED và SERIALIZABLE khi nhiều người dùng cùng truy cập bảng NHANVIEN.

Câu 5. Viết đoạn lệnh sử dụng SAVE TRANSACTION (savepoint) để chỉ rollback một phần thao tác trong một giao dịch gồm nhiều lệnh INSERT/UPDATE trên bảng NHANVIEN và PHONGBAN.

Câu 6. Mô phỏng tình huống deadlock giữa hai giao dịch cùng thao tác trên bảng NHANVIEN và PHONGBAN theo thứ tự ngược nhau, giải thích cách SQL Server phát hiện và xử lý deadlock.

7. Trigger

Câu 1. Viết trigger AFTER INSERT trên bảng NHANVIEN để tự động ghi log (thêm dòng vào bảng lịch sử) mỗi khi có nhân viên mới được thêm.

Câu 2. Viết trigger INSTEAD OF DELETE trên bảng NHANVIEN để ngăn không cho xóa nhân viên khi vẫn còn dữ liệu liên quan ở bảng PHANCONG.

Câu 3. Viết trigger AFTER UPDATE trên bảng NHANVIEN để kiểm tra không cho phép cập nhật MADA thành giá trị âm, nếu vi phạm thì ROLLBACK.

Câu 4. Viết trigger trên bảng PHANCONG để tự động cập nhật (tính toán lại) một giá trị tổng hợp ở bảng NHANVIEN hoặc PHONGBAN mỗi khi có thay đổi dữ liệu ở bảng PHANCONG.

Câu 5. Viết trigger sử dụng bảng ảo inserted và deleted để so sánh giá trị cũ và giá trị mới của MADA khi cập nhật bảng NHANVIEN, chỉ cho phép tăng chứ không cho phép giảm.

Câu 6. Viết trigger DDL (ví dụ: ON DATABASE FOR DROP_TABLE) để ngăn việc xóa nhầm các bảng quan trọng trong cơ sở dữ liệu QL_DeAn.

Câu 7. Viết trigger để đảm bảo ràng buộc nghiệp vụ liên quan đến một nhân viên không được phân công vào hai đề án có khoảng thời gian (TUNGAY–DENNGAY) chồng lấp nhau, tự động chặn hoặc điều chỉnh dữ liệu khi vi phạm.

8. Index (Chỉ mục)

Câu 1. Tạo chỉ mục (index) cho cột khóa ngoại của bảng PHONGBAN tham chiếu đến bảng NHANVIEN nhằm tăng tốc độ JOIN.

Câu 2. Tạo UNIQUE INDEX cho cột HOTEN của bảng NHANVIEN để vừa đảm bảo tính duy nhất vừa tăng tốc tìm kiếm.

Câu 3. Tạo chỉ mục kết hợp (composite index) trên hai cột thường được dùng chung trong điều kiện WHERE của bảng NHANVIEN (ví dụ HOTEN và MADA).

Câu 4. Sử dụng câu lệnh EXPLAIN/hiển thị execution plan (SET STATISTICS IO ON, hoặc xem Actual Execution Plan) để so sánh hiệu năng truy vấn trên bảng NHANVIEN trước và sau khi tạo chỉ mục.

Câu 5. Tạo chỉ mục lọc (filtered index) trên bảng NHANVIEN chỉ đánh chỉ mục cho những dòng thỏa một điều kiện cụ thể.

Câu 6. Trình bày sự khác nhau giữa clustered index và non-clustered index, minh họa bằng cách tạo cả hai loại trên bảng NHANVIEN.

Câu 7. Viết câu lệnh xóa (DROP INDEX) và tạo lại (REBUILD/REORGANIZE) chỉ mục trên bảng NHANVIEN, giải thích khi nào nên bảo trì lại chỉ mục.

9. Role (Vai trò)

Câu 1. Tạo một login mới ở mức SQL Server và một user tương ứng trong cơ sở dữ liệu QL_DeAn.

Câu 2. Tạo một database role (ví dụ role_XemDeAn) chỉ có quyền đọc dữ liệu (SELECT) trên các bảng NHANVIEN và PHONGBAN, sau đó thêm user vừa tạo vào role này.

Câu 3. Tạo một database role (ví dụ role_QuanLyDeAn) có quyền INSERT, UPDATE trên bảng NHANVIEN nhưng không có quyền DELETE.

Câu 4. Sử dụng lệnh sp_addrolemember (hoặc ALTER ROLE ... ADD MEMBER) để gán user vào nhiều role khác nhau, sau đó kiểm tra quyền hạn bằng lệnh EXECUTE AS USER.

Câu 5. Tạo một application role cho ứng dụng quản lý QL_DeAn, minh họa cách kích hoạt application role bằng sp_setapprole.

Câu 6. Viết câu lệnh loại một user ra khỏi role và xóa role khi không còn sử dụng, kiểm tra quyền của user sau khi bị loại khỏi role.

10. Bảo mật (Security)

Câu 1. Viết câu lệnh GRANT cấp quyền SELECT trên bảng NHANVIEN cho một user cụ thể.

Câu 2. Viết câu lệnh DENY quyền DELETE trên bảng NHANVIEN cho một user hoặc role cụ thể, giải thích thứ tự ưu tiên giữa GRANT, DENY và REVOKE.

Câu 3. Viết câu lệnh REVOKE để thu hồi lại quyền đã cấp ở câu 1, kiểm tra lại quyền của user bằng cách đăng nhập thử (hoặc EXECUTE AS).

Câu 4. Cấp quyền EXECUTE cho một stored procedure đã tạo ở phần trước cho một role cụ thể, thay vì cấp quyền trực tiếp trên bảng.

Câu 5. Trình bày và minh họa cách mã hóa dữ liệu nhạy cảm trong bảng NHANVIEN (ví dụ dùng ENCRYPTBYPASSPHRASE/DECRYPTBYPASSPHRASE) đối với cột lưu thông tin cần bảo mật.

Câu 6. Thiết lập column-level security: chỉ cấp quyền SELECT trên một số cột cụ thể của bảng NHANVIEN (không cấp toàn bộ cột) cho một user.

Câu 7. Tạo row-level security (Security Policy với Security Predicate) để mỗi user chỉ xem được các dòng dữ liệu của bảng NHANVIEN thuộc phạm vi quản lý của mình (ví dụ theo PHONGBAN).